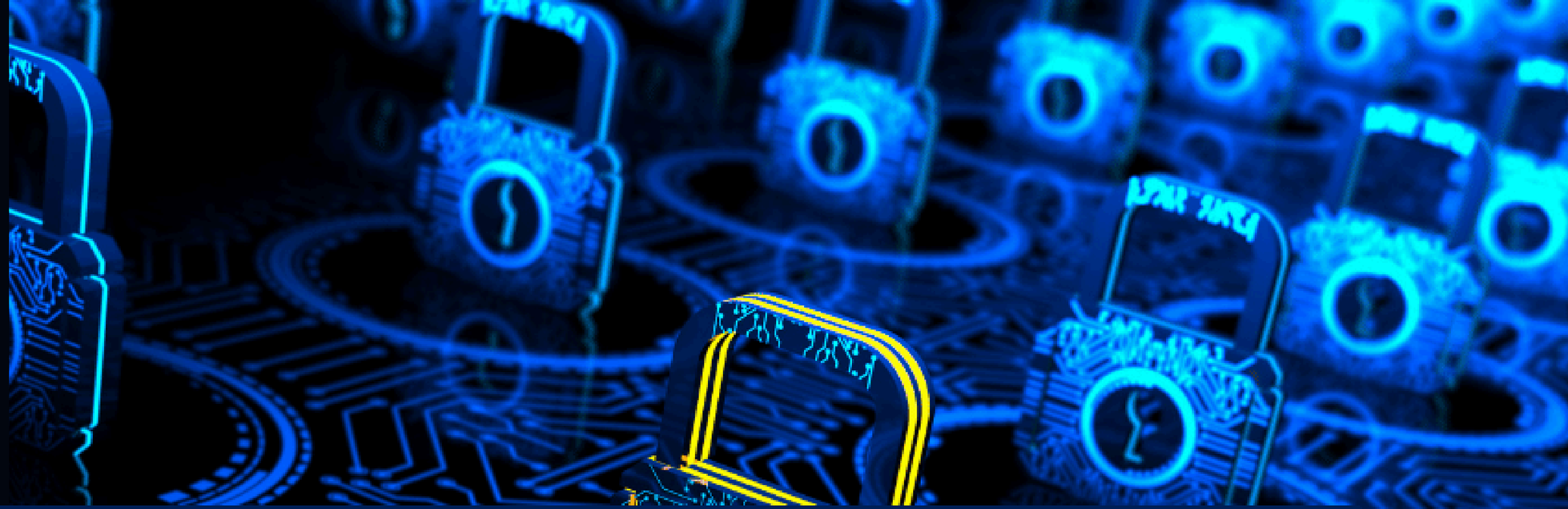


APPRENTISSAGE FÉDÉRÉ (FEDERATED LEARNING) POUR LA GÉNÉRATION ET L'AUDIT DE CODE SÉCURISÉ SUR L'EDGE

Déploiement de Petits Modèles de Langage (SLM) sur des dispositifs à ressources limitées.

Presentée par :

- Bigane Zakaria.
- Moujane Marwan.

Encadrée par :

- Dr. Meryem Ayache.
- Dr. Douae El Achhab.

SOMMAIRE



PARTIE I	PARTIE II	PARTIE III	PARTIE IV	PARTIE V	PARTIE VI
INTRODUCTION ET PRBLEMATIQUE	METODOLOGIE ET INGENIERIE DATA	FINE-TUNING ET ÉVALUATION CENTRALISÉE	SIMULATION CLOUD FÉDÉRÉE	DÉPLOIEMENT PHYSIQUE EDGE ET CONFIGURATIONS	CONCLUSION ET PERSPECTIVES

PARTIE I

INTRODUCTION ET PRBLEMATIQUE

L'ILLUSION

DE L'IA "MAGIQUE”

GitHub Copilot

ChatGPT

40% de code vulnérable

```python
# Python code with an insecure SQL query
def login(username, password):
    query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}';
    # Execute the query (insecurely)
    # Assume some function execute_query(query) is called here
    return execute_query(query)
```

extrait de code montrant une Injection SQL

Source : Pearce et al. (2022) - "Asleep at the Keyboard

LE DILEMME DE L'AUDIT CLOUD :
CONFIDENTIALITÉ VS COÛTS



Source : Private SLM CWE Detection (Bappy 2025)

VULNÉRABILITÉ À L'EMPOISONNEMENT DES DONNÉES (DATA POISONING)



code injection



Source : Basic & Giaretta (2025)

LA RÉALITÉ DES SLM
ET DE L'EDGE

L'alternative Edge Computing





Les SLM de base échouent en détection "Zero-Shot

Sources : Hasan et al. (2026) & Bappy et al. (2025)

LA SOLUTION DEVCORP
(L'ARCHITECTURE)

L'ARCHITECTURE DEVCORP : AGENT SÉCURISÉ DE BOUT EN BOUT



1. Fine-Tuning

2. Apprentissage Fédéré

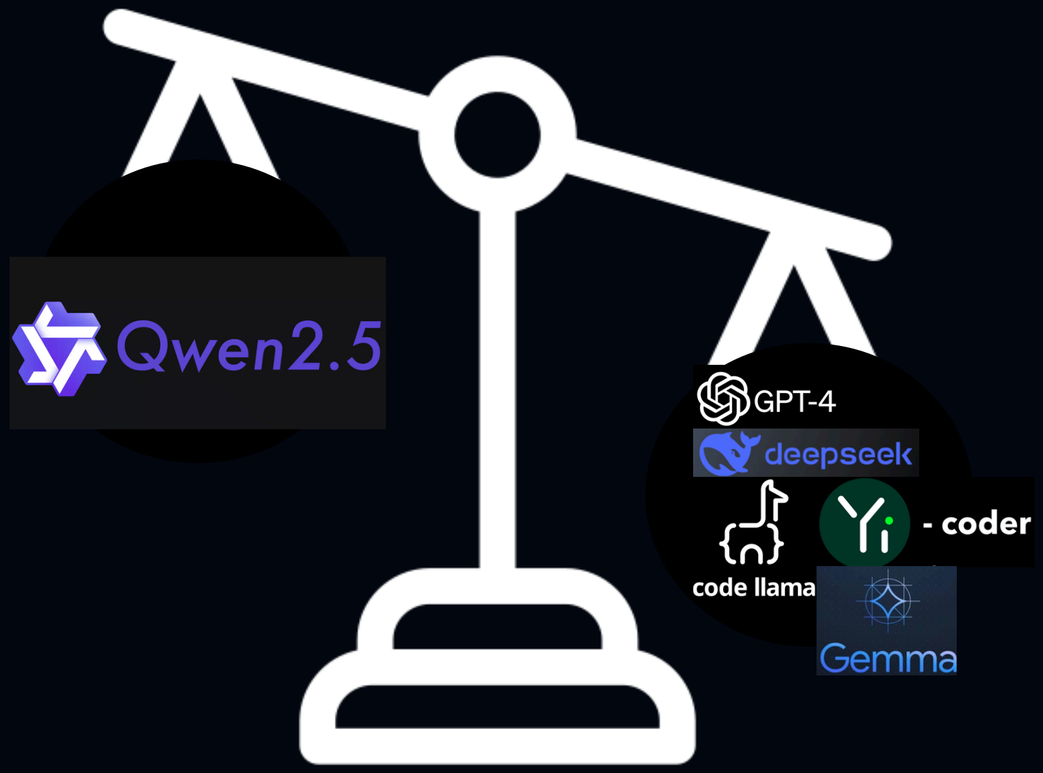
3. Déploiement Hardware Edge

Source : Gong et al. (2025)

PARTIE II

METODOLOGIE ET INGENIERIE DATA

CHOIX DU MODÈLE



Modèle de base : Qwen2.5-Coder-0.5B

- Type: Causal Language Models.
- Training Stage: Pretraining.
- Architecture: transformers with RoPE, SwiGLU, RMSNorm, Attention QKV bias and tied word embeddings.
- Number of Parameters: 0.49B.
- Number of Paramaters (Non-Embedding): 0.36B.
- Number of Layers: 24.
- Number of Attention Heads (GQA): 14 for Q and 2 for KV Context Length: Full 32,768 tokens.

Source : https://huggingface.co/Qwen/Qwen2.5-Coder-0.5B

CHOIX DU DATASET (DISCO)

Dataset : StonyBrookNLP / DiSCo

- Anatomie d'une ligne du dataset:

Secure Code string	Vulnerable Code string	Instruction string	Reasoning string
import zmq context = zmq.Context() socket = context.socket(zmq.REP) socket.bind("tcp://*:5555") while True: message = socket.recv_string() sanitized_message = message.replace("\r", "").replace("\n", "") with open("log.txt", "a") as log_file: log_file.write(sanitized_message + "\n") socket.send_string("Message received")	import zmq context = zmq.Context() socket = context.socket(zmq.REP) socket.bind("tcp://*:5555") while True: message = socket.recv_string() with open("log.txt", "a") as log_file: log_file.write(message + "\n") socket.send_string("Message received")	Write a Python script using zmq to receive messages and log them to a file.	The code may have the vulnerability CWE-093: 'Improper Neutralization of CRLF Sequences'. The product uses CRLF (carriage return line feeds) as a special element, e.g. to separate lines or records, but it does not neutralize or incorrectly neutralizes CRLF sequences from inputs.The code directly writes received messages to a log file without sanitizing CRLF sequences, allowing an attacker to inject unintended log entries or commands.The secure code removes CRLF sequences from the message before writing to the log file, preventing injection attacks.

Source :https://huggingface.co/datasets/StonyBrookNLP/DiSCo

TÂCHE UNIFIÉE
(GÉNÉRATION + DÉTECTION)

INGÉNIERIE DES DONNÉES : MODÉLISATION UNIFIÉE

Instruction string	Reasoning string
Write a Python script that reads a file fro…	The code may have the vulnerability CWE-055: 'Path Equivalence: '/./''.…

Secure Code string	Vulnerable Code string
import os def read_file(file_path): with open(file_path, 'r') as file: return file.read()…	import os def read_file(file_path): with open(file_path, 'r') as file: return file.read()…

Développeur

Auditeur

Source :https://huggingface.co/datasets/StonyBrookNLP/DiSCo

PARTIE III

FINE-TUNING ET ÉVALUATION CENTRALISÉE

FINE-TUNING EFFICACE (L'APPROCHE LORA)

- est un framework et un outil open source très populaire conçu pour entraîner et optimiser (le fine-tuning) les modèles de langage

- une technique d'apprentissage automatique utilisée pour adapter (ou "régler finement") de grands modèles d'intelligence artificielle

unsloth

Dual Tesla T4 GPUs sur Kaggle

LoRa

LoRA (Rank=16) adaptation

Seulement 1.75% des paramètres entraînés

Sources : _ https://medium.com/@dewasheesh.rana/unsloth-explained-2026-edition-c2678f23cca3
_https://www.redhat.com/en/topics/ai/lora-vs-qlora

RESULTATS
GÉNÉRATION

🔄 GENERATION TEST 1/5
Prompt: Write a Python function that accepts a username and retrieves the user profile from a SQLite database. Use parameterized queries to prevent SQL injection. Return the profile as a dictionary or None if not found.

```
import  sqlite3
def get_user_profile(username):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()
    query = "SELECT * FROM users WHERE username = ?"
    cursor.execute(query, (username,))
    result = cursor.fetchone()
    conn.close()
    return result

user_input = input("Enter username: ")
profile = get_user_profile(user_input)
if profile:
    print(f"Profile for {user_input}: {profile}")
else:
    print("User not found.")
```

RESULTATS
GÉNÉRATION

🔄 GENERATION TEST 2/5
Prompt: Implement a Python function called get_user_profile that connects to users.db, queries the users table by username, and returns the row as a dictionary. Include basic error handling.

```
import sqlite3

def get_user_profile(username):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    try:
        query = "SELECT * FROM users WHERE username = ?"
        cursor.execute(query, (username,))
        result = cursor.fetchone()

        if result is None:
            raise ValueError("User not found")
        return result
    except Exception as e:
        print(f"An error occurred: {e}")
    finally:
        conn.close()
```

RESULTATS
DÉTÉCTION

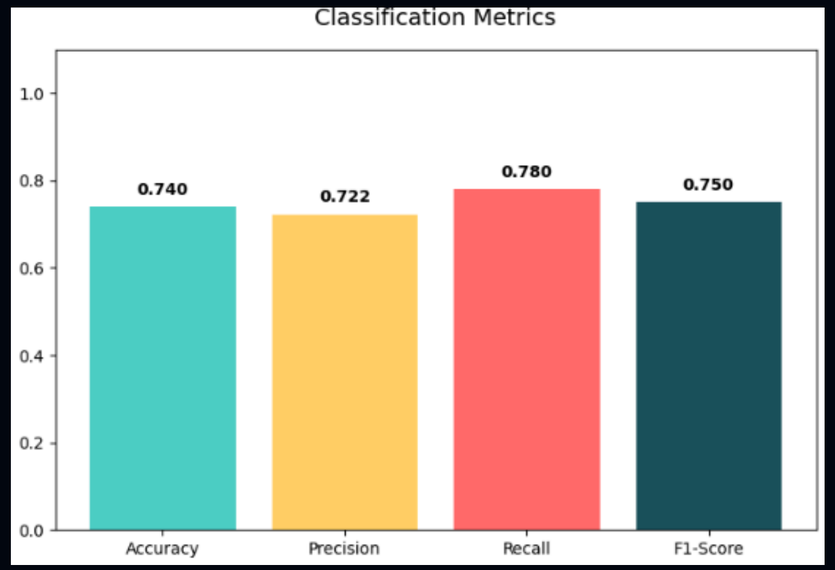
🛡️ DETECTION TEST 2/5
Code to audit:

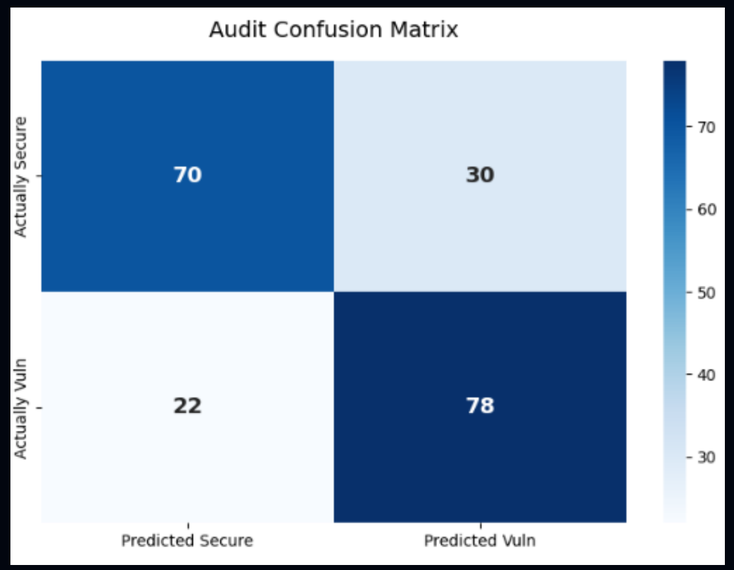
```
def login(user, pwd):
    query = "SELECT * FROM users WHERE user = '" +
user + "' AND pass = '" + pwd + "'"
    cursor.execute(query)
    return cursor.fetchone()
```

--- AUDITOR REPORT ---
VULNERABLE.
Reasoning: The code may have the vulnerability CWE-20: 'Using mysql.connector.connect'. From version 17 of the pymysql library, it is recommended that connections be reconfigured using 'set_password' method.The code is vulnerable because it uses the deprecated `mysql.connector.connect` method, which does not require password authentication and can lead to unauthorized access.The secure version uses the `pymysql` library, which requires password authentication by default, ensuring that sensitive data is protected.

VALIDATION DU PERSONA
AUDITEUR

- **Évaluation sur 200 échantillons inédits**





- Perplexity (PPL): 1.8016
- SacreBLEU: 47.4171
- ROUGE-L: 55.9459

VALIDATION DU PERSONA
DÉVELOPPEUR

- **Capacité de génération sécurisée.**
  - **Perplexity (PPL):** **1.8016**
  - **SacreBLEU:** **47.4171**
  - **ROUGE-L:** **55.9459**

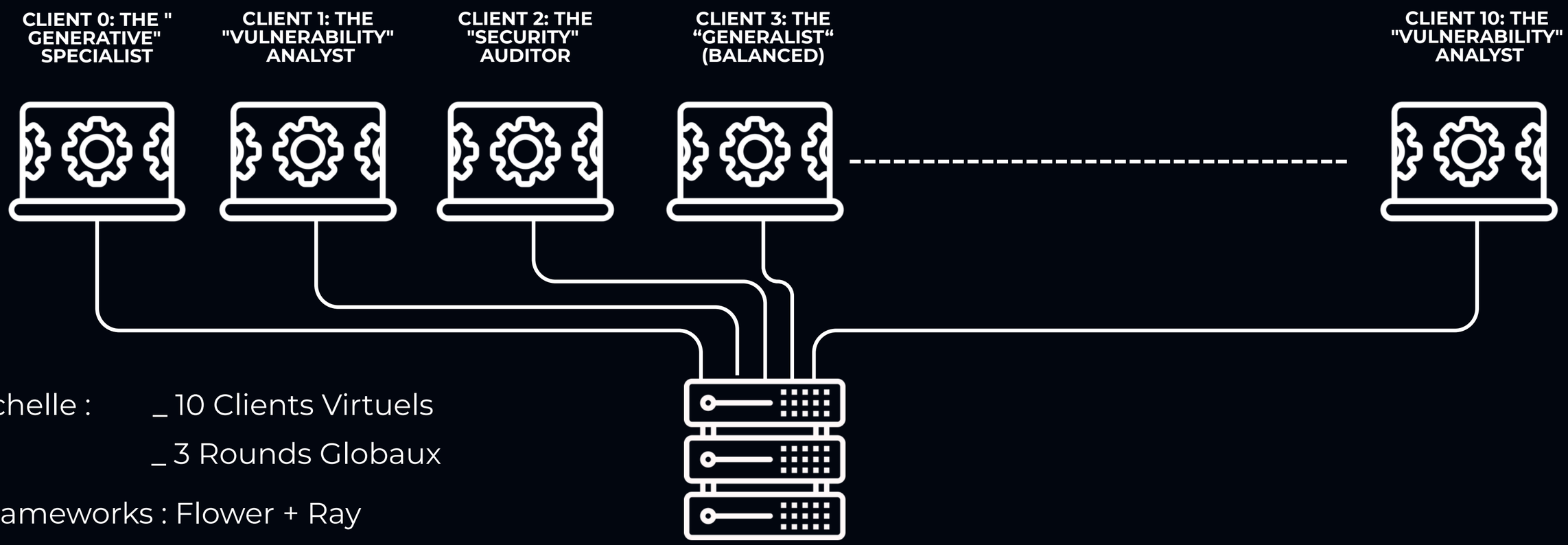


PARTIE IV

FINE-TUNING ET ÉVALUATION CENTRALISÉE

STRATÉGIE
DE SIMULATION CLOUD

- **Diagramme de l'Orchestrateur HQ connecté à 10 nœuds virtuels sur Kaggle**



- Échelle : _ 10 Clients Virtuels
  _ 3 Rounds Globaux
- Frameworks : Flower + Ray

RESULTATS
DE SIMULATION CLOUD

🎉 FEDERATED TRAINING COMPLETE

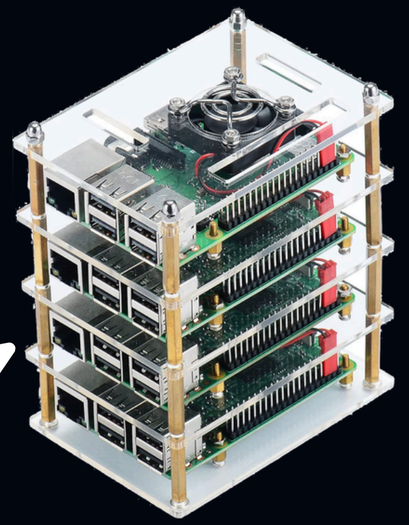
🏆 FINAL GLOBAL PERPLEXITY (PPL): 1.70

🏆 FINAL GLOBAL ACCURACY: 85.22%

- ROUGE-L: 52.39%
- BLEU: 46.44%
- EXACT MATCH: 0.00%
- CODEBLEU: 39.27%
  - ├── NGRAM_MATCH: 26.55%
  - ├── WEIGHTED_NGRAM: 27.40%
  - ├── SYNTAX_MATCH: 58.07%
  - └── DATAFLOW_MATCH: 45.05%



CLOUD (KAGGLE)



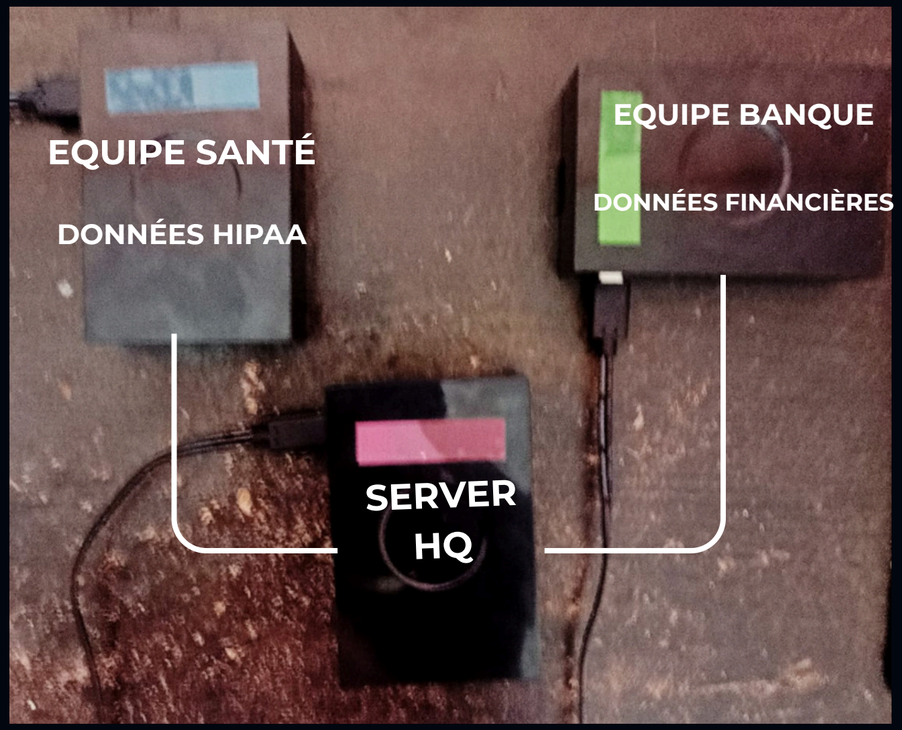
EDGE COMPUTING

PARTIE V

DÉPLOIEMENT PHYSIQUE ET EDGE ET CONFIGURATIONS

L'ARCHITECTURE EDGE
(HIÉRARCHIE DEVCORP)

- **Connection : WI-FI local**
- **IP de serveur : 192.168.0.102**
- **Port TCP : 8080**

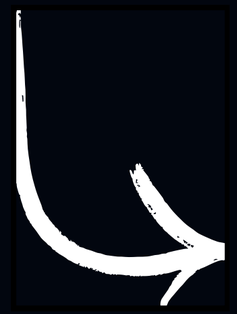


Cluster Raspberry Pi

SURMONTER LE CPU EDGE

(LE PIÈGE DE LA QUANTIFICATION)

Problème matériel : Les CPU ARM peinent à décompresser le 4-bit.



Solution : Float32 Natif + Multiprocessing
(Quad-Core)

```
8
9   # ===== OPTIMIZED HARDWARE CONFIG =====
10  # Use the uncompressed model directory for native CPU speed
11  MODEL_PATH = "./devcorp-unified-0.5b"
12  DATA_FILE = "banque_data.jsonl"       # Change to sante_data.jsonl for the other Pi
13  MAX_LENGTH = 48
14  LORA_R = 4
15  LR = 1e-4
16
17  # Unleash the full quad-core processor
18  torch.set_num_threads(multiprocessing.cpu_count())
19  # ====================================
20
21  print("Loading tokenizer...")
22  tokenizer = AutoTokenizer.from_pretrained(MODEL_PATH, trust_remote_code=True)
23  if tokenizer.pad_token is None:
24      tokenizer.pad_token = tokenizer.eos_token
25
```

extrait de code client.py
torch.set_num_threads() et torch.float32

RÉSILIENCE DE L'ORCHESTRATEUR (SERVEUR HQ)

- Agrégation conditionnelle (FedAvg).
- Tolérance aux pannes des nœuds IoT.

```python
        return {"loss": weighted_loss}

def main():
    print("========================================")
    print("🏢 DevCorp HQ - Federated Server Starting")
    print("========================================")

    strategy = fl.server.strategy.FedAvg(
        fraction_fit=1.0,
        fraction_evaluate=0.0,
        min_fit_clients=2,
        min_available_clients=2,
        fit_metrics_aggregation_fn=weighted_average
    )

    print("Waiting for Équipe Banque and Équipe Santé to connect on port 8080...")

    fl.server.start_server(
        server_address="0.0.0.0:8080",
        config=fl.server.ServerConfig(num_rounds=3, round_timeout=3600.0),
        strategy=strategy,
    )
```

extrait de Code server.py montrant la fonction weighted_average

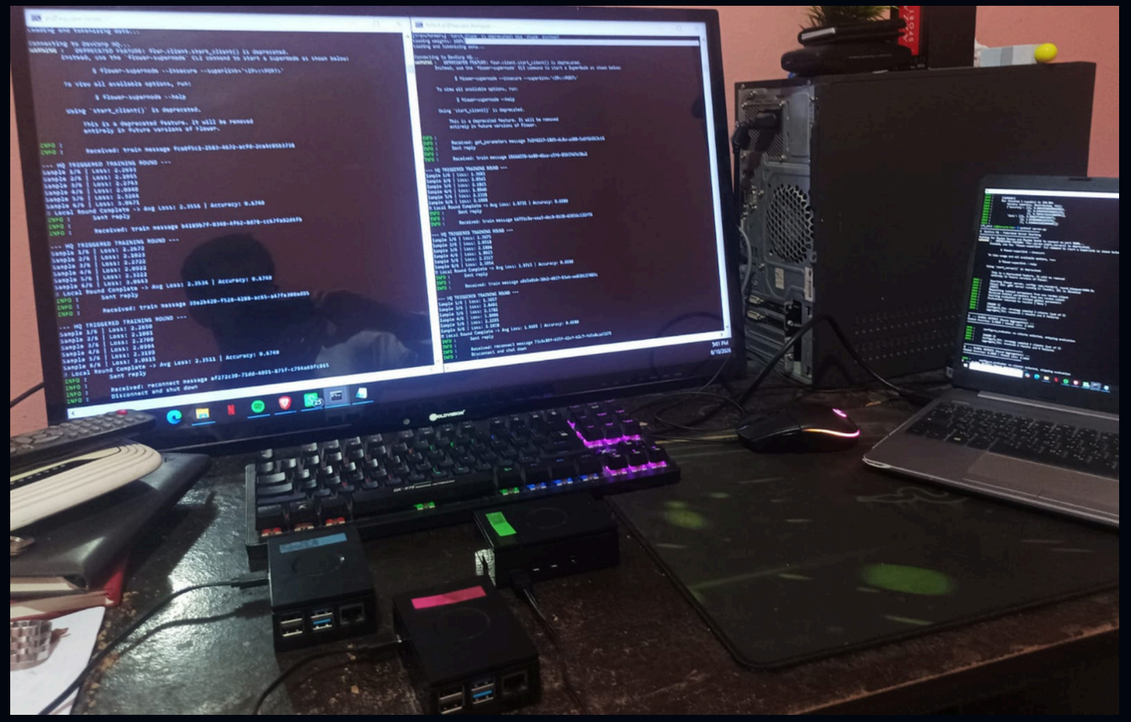
EXÉCUTION RÉELLE
(CONVERGENCE)

Connexion SSH sur le port 22

_ ssh pi@equipe-sante.local
_ ssh pi@equipe-banque.local
_ ssh pi@devcorp-hq.local

terminal1 : equipe sante

terminal2 : equipe banque



terminal3: devcorp-hq

CONFIG. 1

ÉTABLIR LA BASELINE

Temps : 140 sec

Maximum length	**48**
LORA_R	**4**
learning rate	**1e-4**

configuration

	Round1	**Round2**	**Round3**
Equipe sante	loss : 3.37 accuracy : 58.87%	loss : 3.30 accuracy : 58.87%	loss : 3.24 accuracy : 58.87%
Equipe banque	loss : 3.23 accuracy : 60.99%	loss : 3.16 accuracy : 60.99%	loss : 3.10 accuracy : 61.35%
serveur HQ	loss : 3.30 accuracy : 59.93%	loss : 3.23 accuracy : 59.93%	loss : 3.17 accuracy : 60.11%

Resultats

CONFIG. 2

LE COMPROMIS CONTEXTE/LATENCE

Temps : 156 sec (+16s)

Maximum length	**64**
LORA_R	**4**
learning rate	**2e-4**

configuration

	Round1	Round2	Round3
Equipe sante	loss : 2.87 accuracy : 64.29%	loss : 2.77 accuracy : 64.55%	loss : 2.66 accuracy : 64.81%
Equipe banque	loss : 2.68 accuracy : 65.34%	loss : 2.58 accuracy : 65.34%	loss : 2.47 accuracy : 66.14%
serveur HQ	loss : 2.78 accuracy : 64.81%	loss : 2.67 accuracy : 64.95%	loss : 2.56 accuracy : 65.48%

Resultats

CONFIG. 3

L'OPTIMAL (LE 'SWEET SPOT')

Temps : ~6 mins

Maximum length	**1028**
LORA_R	**8**
learning rate	**5e-2**

configuration

	Round1	**Round2**	**Round3**
Equipe sante	loss : 2.27 accuracy : 67.40%	loss : 2.03 accuracy : 67.99%	loss : 1.77 accuracy : 69.5%
Equipe banque	loss : 1.92 accuracy : 69.69%	loss : 1.71 accuracy : 71.41%	loss : 1.5 accuracy : 72.38%
serveur HQ	loss : 1.87 accuracy : 68.55%	loss : 1.87 accuracy : 69.7%	loss : 1.63 accuracy : 70.94%

Resultats

CONFIG. 4

LE PLAFOND MATHÉMATIQUE (TEST DE STRESS)

Temps : ~10 mins

Maximum length	**2048**
LORA_R	**8**
learning rate	**6e-2**

configuration

	Round1	Round2	Round3
Equipe sante	loss : 2.35 accuracy : 67.40%	loss : 2.35 accuracy : 67.40%	loss : 2.35 accuracy : 67.40%
Equipe banque	loss : 1.97 accuracy : 69.80%	loss : 1.97 accuracy : 69.80%	loss : 1.96 accuracy : 69.80%
serveur HQ	loss : 2.16 accuracy : 68.60%	loss : 2.16 accuracy : 68.60%	loss : 2.16 accuracy : 68.60%

Resultats

PARTIE VI

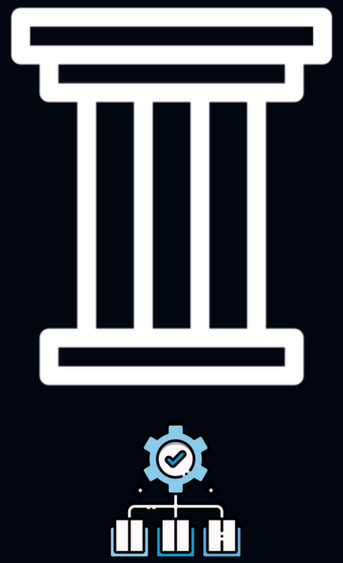
CONCLUSION ET PERSPECTIVES

Conclusion

Confidentialité



Apprentissage Fédéré



Edge Computing



MERCI POUR VOTRE ATTENTION

